

Frontend Advanced

Next.js 풀스택 실습

Next.js 핵심 기능 살펴보기

목차 Next.js 풀스택 실습

1. Next.js 핵심 기능 살펴보기
2. 프로젝트 구조 및 기능 이해하기
3. [실습] Fetch 기반 검색 기능 구현하기
4. [실습] axios로 검색 기능 리팩토링하기
5. [실습] 검색 기능 고도화하기

Next.js 핵심 기능 살펴보기

01

Next.js 핵심 개념

App Router · Server/Client
Component · 캐싱

02

프로젝트 구조 및 기능 이해

디렉토리 구조 및 기능 확인

03

[실습] Fetch 기반 검색 기능

Direct Fetch와 Route Handler
기반 검색 기능 구현

04

[실습] axios 리팩토링

fetch를 axios로 리팩토링

05

[실습] 검색 기능 고도화

URL 쿼리 파라미터 기반
서버 필터링 구현

Next.js로 검색 기능을 구현하며
풀스택 흐름 이해하기



NEXT.js

높은 퀄리티의 풀스택 웹 애플리케이션을 제작할 수 있는

React 기반의 프레임워크



Routing과 Rendering

Data Fetching

Caching

Optimizing

프레임워크를 잘 사용하려면, 프레임워크의 흐름을 관통하는 핵심 개념을 이해해야 한다.

Next.js의 핵심 개념을 크게 4가지로 나누어보면 위와 같다.



Next.js의 4가지 핵심 요약

1 Routing & 렌더링

- 폴더 = URL 구조의 파일 기반 라우팅
- 기본은 Server Component로 렌더링

2 Data Fetching

- 서버에서 직접 fetch · Route Handler · Server Action 등 여러 방식 제공

3 Caching

- Data Cache · Full Route Cache · Router Cache 등 다층 캐싱 (v15부터 fetch는 기본 비캐싱)

4 Optimizing

- Image · Link · Font · Script · Metadata 등 내장 최적화 컴포넌트



프로젝트 구성 살펴보기

구분	기술
백엔드	FastAPI, SQLAlchemy(ORM), SQLite, Pydantic(v2)
프론트엔드	Next.js (v15+), Typescript, Tailwind CSS (v4)
HTTP 클라이언트	fetch (Server Component)

↓ 로컬에서 실행해보고 싶다면, 아래 명령어로 저장소를 클론하세요

```
$ git clone https://github.com/elice-contents/next.js_fullstack_boilertemplate.git
```



프로젝트 살펴보기

폴더 구조

```
backend/  
├─ __pycache__/  
├─ blog.db      # SQLite DB 파일  
├─ main.py      # FastAPI 엔트리포인트  
└─ pyproject.toml # Python 패키지  
  
frontend/  
├─ app/  
│   ├── posts/  
│   │   ├── [postId]/      # 동적 라우트  
│   │   │   ├── edit/  
│   │   │   │   └─ page.tsx  # 수정 페이지  
│   │   │   └─ DeleteButton.tsx # use client  
│   │   └─ page.tsx      # 상세 페이지  
│   ├── new/  
│   │   └─ page.tsx      # 작성 페이지  
│   ├── error.tsx        # 에러 UI (특수 파일)  
│   ├── loading.tsx      # 로딩 UI (특수 파일)  
│   └─ page.tsx          # 목록 페이지  
├─ actions.ts            # Server Actions  
├─ globals.css  
├─ layout.tsx            # 공통 레이아웃  
└─ page.tsx              # 홈  
  
next-env.d.ts  
next.config.mjs          # Next.js 설정  
package.json  
postcss.config.mjs  
tsconfig.json
```

App Router → URL 매핑

app/page.tsx



/

app/posts/page.tsx



/posts

app/posts/new/page.tsx



/posts/new

app/posts/[postId]/page.tsx



/posts/123

app/posts/[postId]/edit/page.tsx



/posts/123/edit

하나의 저장소에서 **backend/**, **frontend/**로 명확히 분리된 구조로 현업 표준과 동일하다



Method	Endpoint	설명
GET	/posts	전체 글 목록 조회
GET	/posts/{id}	특정 글 상세 조회
POST	/posts	새 글 작성
PUT	/posts/{id}	글 수정
DELETE	/posts/{id}	글 삭제

frontend 디렉토리 하위의 **actions.ts**는 서버 함수를 한 파일에서 모아서 관리하고 있다.

backend 디렉토리 하위의 **main.py**에서 실제 기능이 구현되어 있다.



프로젝트 실행하기

frontend 실행하기

~/next.js_fullstack_boiler
template/

```
$ cd frontend
```

/frontend 디렉토리로 이동

~/next.js_fullstack_boiler
template/frontend

```
$ npm install
```

패키지 설치

```
$ touch .env.local
```

.env.local 생성

```
$ echo FASTAPI_URL='http://localhost:8000' >> .env.local # .env.local 환경변수 추가  
$ echo BASE_PATH='/proxy/3000' >> .env.local  
$ echo ASSET_PREFIX='/proxy/3000' >> .env.local
```

```
$ npm run dev
```

서버 실행



프로젝트 실행하기

backend 실행하기

~/next.js_fullstack_boiler
template/

```
$ cd backend
```

/backend 디렉토리로 이동

~/next.js_fullstack_boiler
template/backend

```
$ uv sync
```

#lock 파일 기준으로 venv 생성 및 설치

```
$ uv run fastapi dev main.py
```

서버 실행



프로젝트 실행하기

The screenshot shows the VS Code interface. The terminal panel on the left contains the following text:

```
(.venv) elicer@7d8a7cfd295f:~/next.js_fullstack_boilertemplate/backend$ uv sync  
(.venv) elicer@7d8a7cfd295f:~/next.js_fullstack_boilertemplate/backend$ uv run fastapi dev main.py  
INFO: Will watch for changes in these directories: ['/home/elicer/next.js_fullstack_boilertemplate/backend']  
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)  
INFO: Started reloader process [2202] using WatchFiles  
INFO: Started server process [2220]  
INFO: Waiting for application startup.  
INFO: Application startup complete.  
INFO: 106.253.185.239:0 - "GET / HTTP/1.1" 404 Not Found  
INFO: 127.0.0.1:38330 - "GET /posts HTTP/1.1" 200 OK
```

The file explorer on the right shows a folder structure with two items: `npm frontend` and `python backend`. The `python backend` folder is selected. The status bar at the bottom indicates the current file is `main*` at line 2, column 1, with 4 spaces, UTF-8 encoding, and LF line endings.

두 개의 터미널을 띄워두고, 각각 frontend, backend를 실행할 수 있다.



frontend/app/actions.ts 살펴보기

```
"use server";

import { revalidateTag } from "next/cache";
import { redirect } from "next/navigation";

// ——— 게시물 생성 ———
export async function createPost(formData: FormData) {
  const title = formData.get("title") as string;
  const content = formData.get("content") as string;

  const res = await fetch(`${process.env.FASTAPI_URL}/posts`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title, content }),
  });

  if (!res.ok) {
    const error = await res.json();
    throw new Error(error.detail ?? "게시글 생성에 실패했습니다");
  }

  revalidateTag("posts-list");
  redirect("/posts");
}
```

- **actions.ts**는 클라이언트 컴포넌트에서 직접 호출할 수 있는 **Server Actions**를 정의하는 파일
- API 엔드포인트를 별도로 만들지 않고도 서버 측 데이터 변경 로직을 처리한다.
- Next.js v15 이전 버전을 사용하는 경우, **fetch** 결과를 캐시에 저장해 매번 FastAPI를 호출하지 않으므로, 작업 후 캐시를 그대로 두면 이전 데이터가 화면에 남아있음
- 요청 성공 시 **revalidateTag**로 특정 캐시 태그를 선택해서 캐시를 무효화하고, **redirect**로 지정한 페이지로 이동하도록 구현함



Next.js 15+ fetch 캐싱 업데이트

Next.js 14

```
const res = await  
fetch('http://localhost:8000/posts')
```

Next.js 15

```
const res = await  
fetch('http://localhost:8000/posts',  
{  
  cache: 'force-cache',  
  next: { tags: ['posts-list'] }  
})
```

Next.js 16

```
export async function getPosts() {  
  'use cache'  
  cacheLife('hours')  
  cacheTag('posts-list')  
  return  
  fetch('http://localhost:8000/posts').  
  then(r => r.json())  
}
```

16버전부터 **fetch** 캐시는 **use cache**와 같이 명시적으로 선언하는 방식으로 바뀌었고,
어디서 어떻게 캐시할 지 **개발자가 직접 제어**하는 구조로 변경되었다.

Next.js 16 revalidateTag 업데이트

Next.js 14/15

```
"use server";

import { revalidateTag } from "next/cache";
import { redirect } from "next/navigation";

// ——— 게시물 생성 ———
export async function createPost(formData: FormData) {
  const title = formData.get("title") as string;
  const content = formData.get("content") as string;

  const res = await fetch(`${process.env.FASTAPI_URL}/posts`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title, content }),
  });

  if (!res.ok) {
    const error = await res.json();
    throw new Error(error.detail ?? "게시글 생성에 실패했습니다");
  }

  revalidateTag("posts-list");
  redirect("/posts");
}
```

Next.js 16

```
"use server";
import { revalidateTag } from "next/cache";
import { redirect } from "next/navigation";

export async function createPost(formData: FormData) {
  const title = formData.get("title") as string;
  const content = formData.get("content") as string;

  const res = await fetch(`${process.env.FASTAPI_URL}/posts`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title, content }),
  });

  if (!res.ok) {
    const error = await res.json();
    throw new Error(error.detail ?? "게시글 생성에 실패했습니다");
  }

  revalidateTag("posts-list", "max");
  redirect("/posts");
}
```

Next.js 16부터 필수로 **revalidateTag**의 두 번째 인수로 **'max'**, **'hours'** 또는 **{ expire: 3600 }** 같은 **cacheLife** 프로필을 전달해야 한다.



데이터 구조 살펴보기

Post

```
class Post(Base):
    __tablename__ = "posts"
    id = Column(Integer,
primary_key=True, index=True)
    title = Column(String(200),
nullable=False)
    content = Column(Text,
nullable=False)
    created_at = Column(DateTime,
default=lambda:
datetime.now(timezone.utc))
```

PostCreate

```
class PostCreate(BaseModel):
    title: str
    content: str

    @field_validator("title")
    @classmethod
    def title_not_empty(cls, v: str) -> str:
        if not v.strip():
            raise ValueError("제목은 비워둘 수 없
습니다")
        return v.strip()
```

PostResponse

```
class PostResponse(BaseModel):
    id: int
    title: str
    content: str
    created_at: datetime
    model_config = {"from_attributes": True}
```

- **main.py**에는 DB 테이블 구조를 정의하는 **SQLAlchemy** 모델과, 요청/응답 데이터의 형태와 유효성을 정의하는 **Pydantic** 스키마가 선언되어 있다.
- API 호출 시 **Pydantic** 스키마를 기준으로 데이터를 검증하고 직렬화하여 주고받는다.



게시물 생성 요청/응답 성공 흐름



- 요청 데이터를 서버에 전송하면 **유효성 검사** → **DB 저장** 순으로 처리된다.
- 성공 시 서버가 생성된 객체를 반환하여 브라우저에서 확인할 수 있다.



게시물 생성 요청/응답 실패 흐름

DB 오류 발생 시 → 500 반환



유효성 검사 실패 시 → 422 반환



- 유효성 검사 실패 시 즉시 **422**를 반환한다.
- DB 오류 시엔 **rollback** 후 **500**을 반환하며, 두 경우 모두 데이터는 저장되지 않는다.



Server Component

frontend/app/posts/page.tsx 살펴보기

```
# posts/page.tsx

export default async function PostsPage() {
  const res = await fetch(`${process.env.FASTAPI_URL}/posts`, {
    next: { tags: ["posts-list"] },
  });

  if (!res.ok) {
    throw new Error("게시글 목록을 불러오는 데 실패했습니다");
  }

  const posts: Post[] = await res.json();

  return (
    <main>
      ...
      {posts.length === 0 ? (
        <p className="text-center text-gray-400 py-20">
          아직 작성된 게시글이 없습니다.
        </p>
      ) : (
        <ul className="space-y-3">
          {posts.map((post) => (
            <li key={post.id}>
              ...
            </li>
          ))}
        </ul>
      )}
    </main>
  );
}
```

- **use client**를 쓰지 않아 기본적으로 **Server Component**가 적용됨
- 브라우저 요청 시 **PostsPage** 함수가 실행되어 **완성된 HTML**을 브라우저로 전송함
- **Server Component**에서는 브라우저에서 값의 변화를 추적하는 **State**가 구조적으로 불필요함



Server Component vs Client Component

Server Component와 Client Component 비교

	Server Component	Client Component
선언	없음	Use client 추가
함수	Async function	function
데이터	Await fetch	useEffect 안에서 fetch
저장	변수	useState
로딩	없음 (서버에서 완료)	Loading State 필요
에러	Error.tsx	State에서 조건부 렌더링
CORS	불필요	필요



Server Component vs Client Component

Server Component vs Client Component 선택 기준 예시

화면	인터랙션	컴포넌트
게시글 목록	-	Server
게시글 상세	-	Server
게시글 삭제	클릭 이벤트	Client
게시글 작성 폼	입력 및 제출 이벤트	Client / Server
로딩/에러	상태 변화	Client

“이 컴포넌트에 사용자 인터랙션이 필요한가?”를 고민해보고,
필요하다면 **Client Component**, 필요하지 않다면 **Server Component**를 사용하는 것이 적절하다.



Server Component vs Client Component

Component 선택 기준 예시

화면	인터랙션	컴포넌트
게시글 목록	-	Server
게시글 상세	-	Server
게시글 삭제	클릭 이벤트	Client
게시글 작성 폼	입력 및 제출 이벤트	Client / Server
로딩/에러	상태 변화	Client

- 게시물 작성 폼의 경우, `useState`, `onChange`, `onSubmit` 없이 HTML 폼 + Server Action 조합만으로 제출이 가능하기 때문에 **Server Component**로 구현 가능하다.
- Client Component**는 실시간 유효성 검사나 로딩 상태 표시가 필요할 때 적용 가능하다.

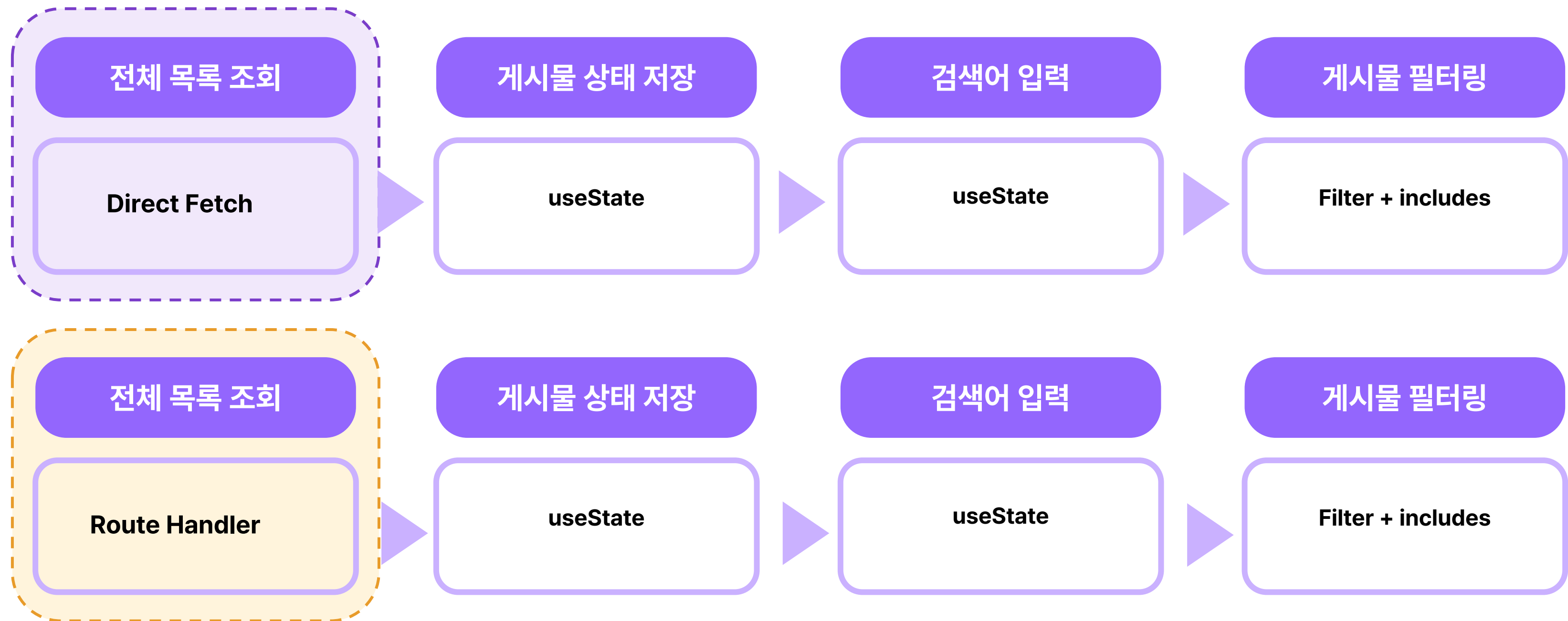
[실습] Fetch 기반 검색 기능 구현하기

Client Component를 활용해 게시물 검색 구현하기

구분	기술
백엔드	FastAPI, SQLAlchemy(ORM), SQLite, Pydantic (v2)
프론트엔드	Next.js (v15), Typescript, Tailwind CSS (v4)
HTTP 클라이언트	fetch (Server Component), fetch (Client Component)

[실습] Fetch 기반 검색 페이지 구현하기

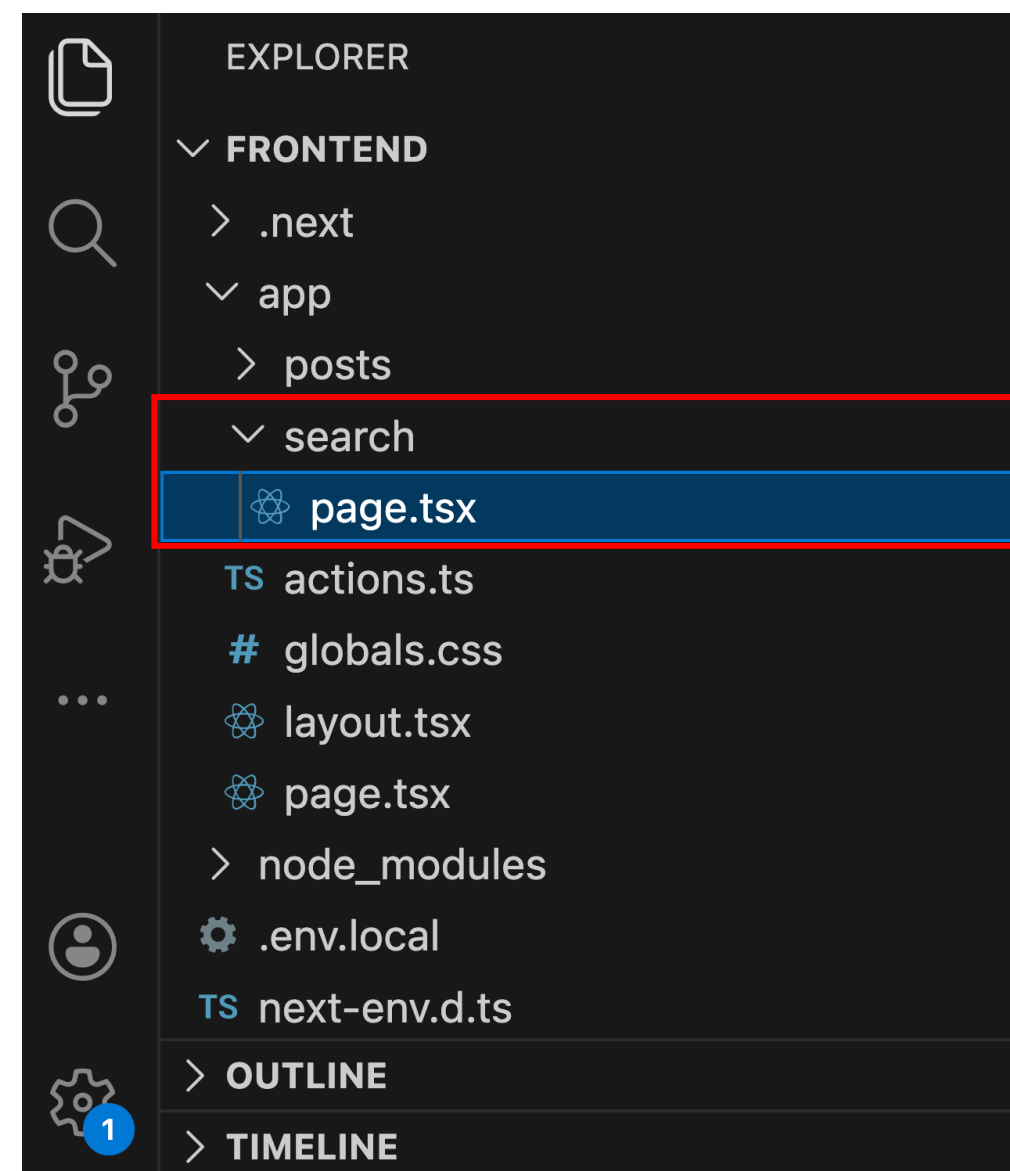
Direct Fetch와 Route Handler 방식 모두 사용해보기



- **Direct Fetch** Client Component에서 외부 API로 직접 **fetch**해 전체 게시물을 가져온다
- **Route Handler** Client Component에서 **/api/search** 라우트를 거쳐 서버에서 외부 API를 호출한다

[실습] Fetch 기반 검색 페이지 구현하기 (Direct Fetch)

파일 생성 및 useState로 상태 선언 (query, results, loading, error)



frontend/app/search/page.tsx

```
"use client";

import { useState, useEffect } from "react";
import Link from "next/link";

type Post = {
  id: number;
  title: string;
  content: string;
  created_at: string;
};

export default function SearchPage() {
  const [query, setQuery] = useState("");
  const [results, setResults] = useState<Post[]>([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);
  ...
```

- frontend > app > search > page.tsx를 생성한다.
- use client로 Client Component 를 선언한다.
- SearchPage 함수에 query, results, loading, error에 해당하는 state 선언 및 초기화한다.



[실습] Fetch 기반 검색 페이지 구현하기 (Direct Fetch)

UseEffect 안에서 게시물 결과물 fetch

frontend/app/search/page.tsx

```
useEffect(() => {  
  async function fetchPosts() {  
    setLoading(true);  
    setError(null);  
    try {  
      const base = "http://localhost:8000";  
      const res = await fetch(`${base}/posts`);  
  
      if (!res.ok) {  
        throw new Error("게시글 목록을 불러오는 데 실패했습니다");  
      }  
  
      const data: Post[] = await res.json();  
      setResults(data);  
    }  
    ...  
  }  
}
```

- **useEffect** 안에서 **async** 함수를 별도로 정의한 뒤 즉시 호출하는 패턴을 적용한다.
- **Client Component**이므로 브라우저가 직접 **FastAPI**를 호출한다.
- 요청 시작 시, 로딩 상태와 에러 상태를 변경한다.



[실습] Fetch 기반 검색 페이지 구현하기 (Direct Fetch)

catch와 finally 블록에서 에러와 로딩 상태 업데이트

frontend/app/search/page.tsx

```
useEffect(() => {  
  ...  
  catch (err) {  
    setError(  
      err instanceof Error ? err.message : "알 수 없는 오류가 발생했습니다"  
    );  
  } finally {  
    setLoading(false);  
  }  
}  
  
fetchPosts();  
}, []);
```

- **catch** 블록에서 에러 상태를 업데이트한다.
- **finally**로 성공·실패 시에 공통으로 실행되는 로딩 상태를 업데이트한다.



[실습] Fetch 기반 검색 페이지 구현하기 (Direct Fetch)

실시간 검색 필터링 구현

frontend/app/search/page.tsx

```
const filtered = results.filter(post =>
  post.title.toLowerCase().includes(query.toLowerCase()) ||
  post.content.toLowerCase().includes(query.toLowerCase())
);
```

- **results**에는 저장된 게시물 전체 목록이 저장된다.
- **filtered**에는 검색어가 바뀔 때마다 재계산되는 필터링된 게시물 목록이 저장된다.
- **toLowerCase**로 대소문자 구분 없이 검색하도록 한다.



[실습] Fetch 기반 검색 페이지 구현하기 (Direct Fetch)

.env.local에 Client Component에서도 접근 가능한 환경 변수 추가

frontend/.env.local

NEXT_PUBLIC_FASTAPI_URL=/proxy/8000

frontend/app/search/page.tsx

```
useEffect(() => {
  setLoading(true);
  setError(null);
  fetch(`${process.env.NEXT_PUBLIC_FASTAPI_URL}/posts`)
    .then((res) => {
      if (!res.ok) throw new Error("게시글을 불러오는 데 실패했습니다");
      return res.json();
    })
    .then((data: Post[]) => setResults(data))
    .catch((err: Error) => setError(err.message))
    .finally(() => setLoading(false));
}, []);
```

- .env.local 파일에 NEXT_PUBLIC으로 시작하는 환경변수를 추가하여 브라우저(클라이언트) 코드에서 사용할 값을 Next.js에 명시
- page.tsx 파일에서 추가한 환경변수 반영



CORS (Cross-Origin Resource Sharing)

- 브라우저가 다른 출처 (Origin)의 서버에 요청할 때 적용되는 보안 정책으로, 다른 출처의 리소스 접근을 기본적으로 차단한다.
- 서버가 명시적으로 허용하지 않으면 응답이 와도 브라우저가 읽지 못하게 막는다.
- 서버에서 CORS (Cross-Origin Resource Sharing)를 설정하여 통신을 허용할 수 있다.
- Origin은 Protocol + Host + Port으로 구성된다.
 - Next.js : http://localhost:3000
 - FastAPI : http://localhost:8000

CORS (Cross-Origin Resource Sharing)

Next.js: 3000

Server Component	fetch(FASTAPI_URL/posts)	FastAPI: 8000	blog.db
Client Component	Route Handler	fetch(FASTAPI_URL/search)	FastAPI: 8000

Server Component는 서버 간 통신으로 **CORS** 설정이 불필요하지만,
Client Component는 브라우저가 요청하므로 백엔드에서 **CORS** 설정이 필수이다.

CORS (Cross-Origin Resource Sharing)

백엔드 코드에서 CORS 미들웨어 옵션을 설정하여 허용할 수 있다.

backend/main.py

```
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=["http://localhost:3000"],  
    allow_credentials=True,  
    allow_methods=["*"],  
    allow_headers=["*"],  
)
```

허용할 출처 명시

쿠키/인증 헤더 포함 요청 허용

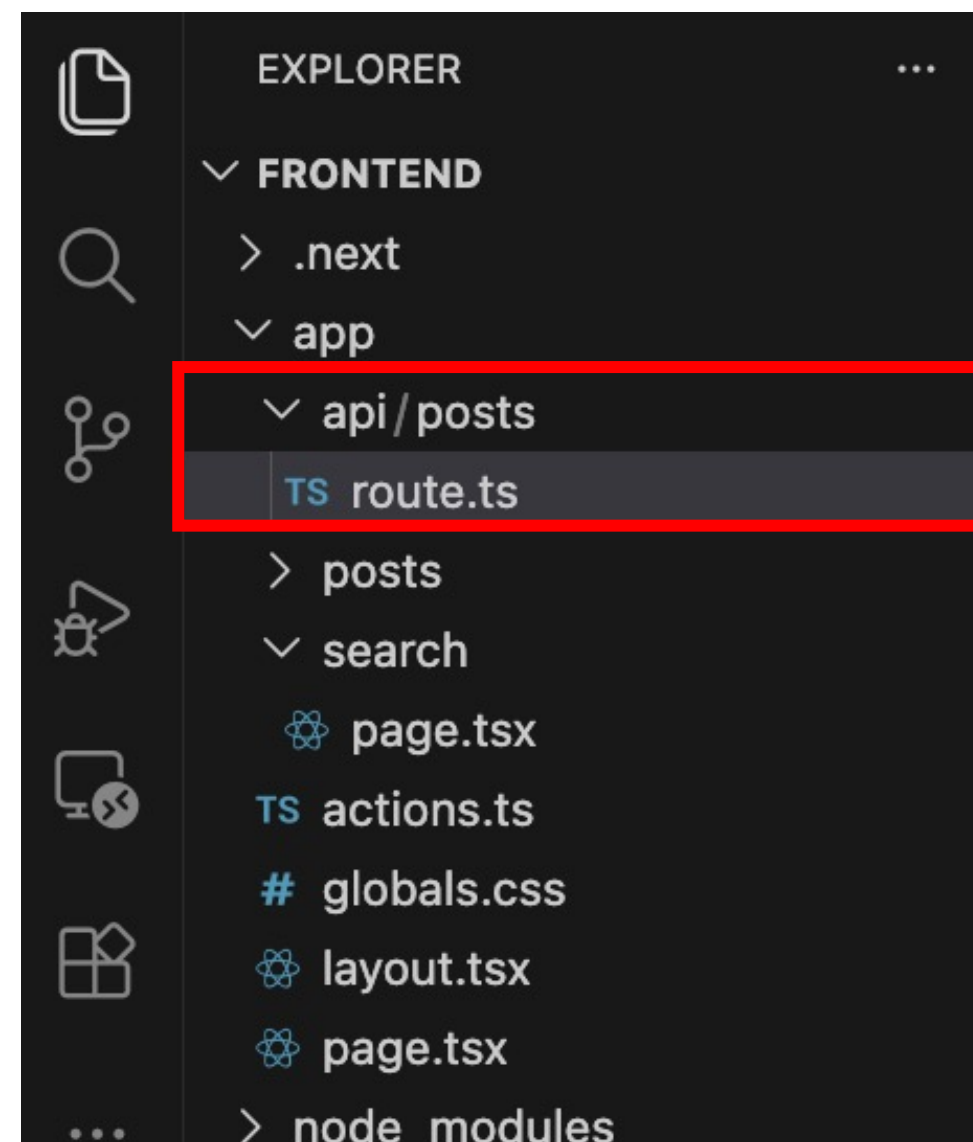
GET, POST, PUT, DELETE 등 모든 메서드 허용

Content-Type 등 모든 헤더 허용

<http://localhost:3000>에서 오는 요청에 한해, 인증 포함 모든 메서드와 헤더를 허용하는 설정이다.



[실습] Fetch 기반 검색 페이지 구현하기 (Route Handler)



- Frontend > app > api > posts > route.ts를 생성한다.
- 브라우저 → Route Handler → FastAPI 흐름으로 동작하도록 구현한다.
- Page.tsx 호출 방식 (백엔드 URL 직접 호출 → /api/posts 호출로 변경)을 변경한다.



[실습] Fetch 기반 검색 페이지 구현하기 (Route Handler)

route.ts의 GET 함수 구현

frontend/app/api/search/route.ts

```
export async function GET() {  
  // ...  
  const res = await fetch(`${fastapiUrl}/posts`);  
  
  if (!res.ok) {  
    return NextResponse.json(  
      { error: "게시글을 불러오는 데 실패했습니다" },  
      { status: res.status }  
    );  
  }  
  
  const data = await res.json();  
  return NextResponse.json(data);  
}
```

- GET /api/search 요청을 받아 FastAPI를 대신 호출하는 **Route Handler**를 구현한다.
- 서버에서 실행되므로 **NEXT_PUBLIC** 접두사 없는 **FASTAPI_URL** 환경변수를 사용한다.



[실습] Fetch 기반 검색 페이지 구현하기 (Route Handler)

page.tsx 호출 방식 수정

BEFORE

기존: 백엔드 URL 직접 호출 (http://localhost:8000)

```
useEffect(() => {
  setLoading(true);
  setError(null);

  fetch(`${process.env.NEXT_PUBLIC_FASTAPI_URL}/posts`)
    .then((res) => {
      if (!res.ok) throw new Error("게시글을 불러오는 데 실패했습니다");
      return res.json();
    })
    .then((data: Post[]) => setResults(data))
    .catch((err: Error) => setError(err.message))
    .finally(() => setLoading(false));
}, []);
```



AFTER

변경: Route Handler 경우 (/api/search)

```
useEffect(() => {
  setLoading(true);
  setError(null);

  fetch(`${BASE_PATH}/api/search`)
    .then((res) => {
      if (!res.ok) throw new Error("게시글을 불러오는 데 실패했습니다");
      return res.json();
    })
    .then((data: Post[]) => setResults(data))
    .catch((err: Error) => setError(err.message))
    .finally(() => setLoading(false));
}, []);
```

Route Handler (/api/search)를 호출하도록 구현한다.



[실습] Fetch 기반 검색 페이지 구현하기 (Route Handler)

.env.local 파일 내 환경변수 수정

BEFORE

기존: NEXT_PUBLIC_FASTAPI_URL 환경변수 사용

```
NEXT_PUBLIC_FASTAPI_URL=/proxy/8000
```

AFTER

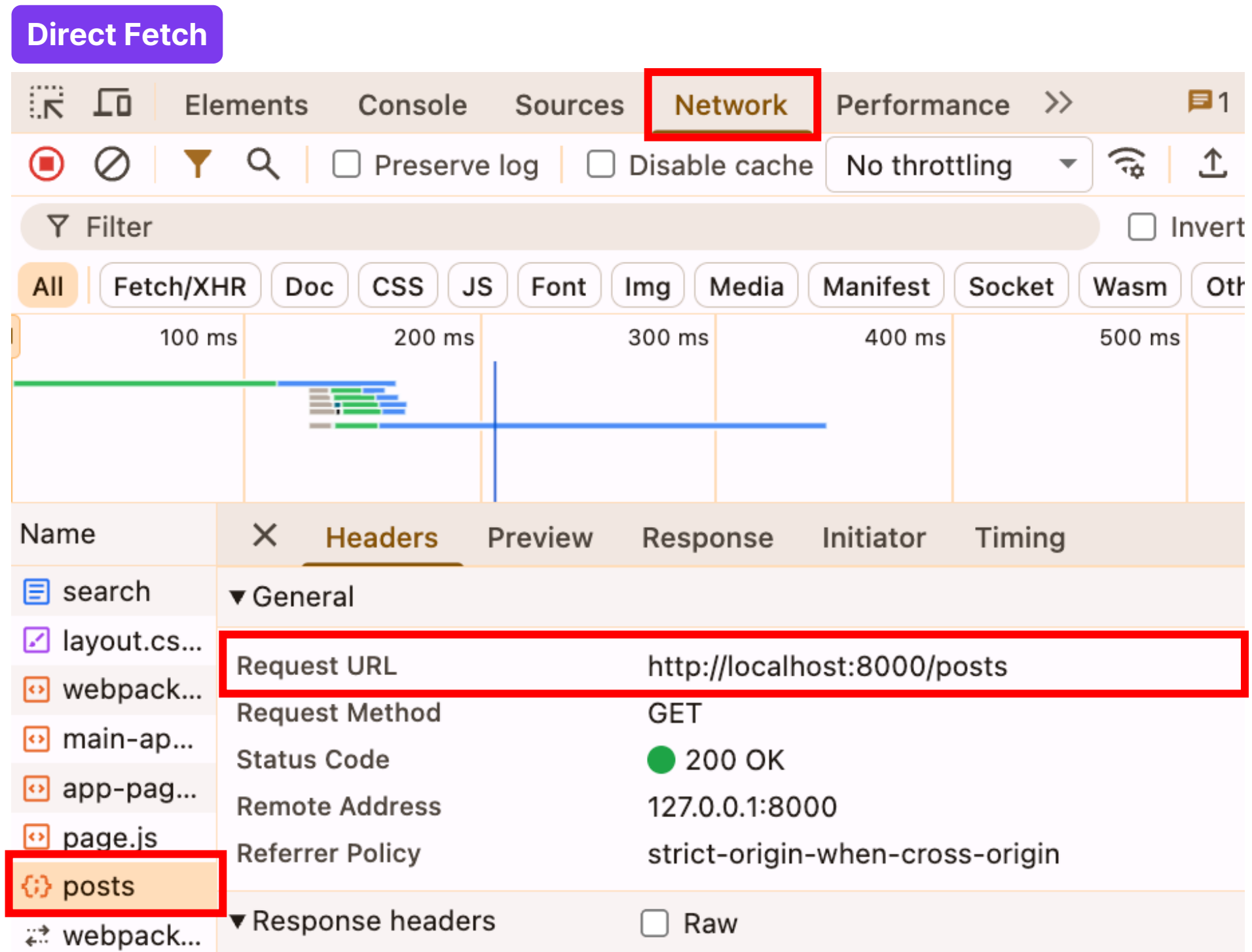
변경: NEXT_PUBLIC_FASTAPI_URL 환경변수 미사용

```
FASTAPI_URL=http://localhost:8000  
# NEXT_PUBLIC_FASTAPI_URL=/proxy/8000
```

Route Handler는 서버에서만 동작하므로, 클라이언트에 값을 노출할 필요가 없어
보안 상 **NEXT_PUBLIC_**으로 시작하는 환경변수를 사용하지 않아도 된다.

Direct Fetch, Route Handler 구현 방식에 따른 Request URL 비교

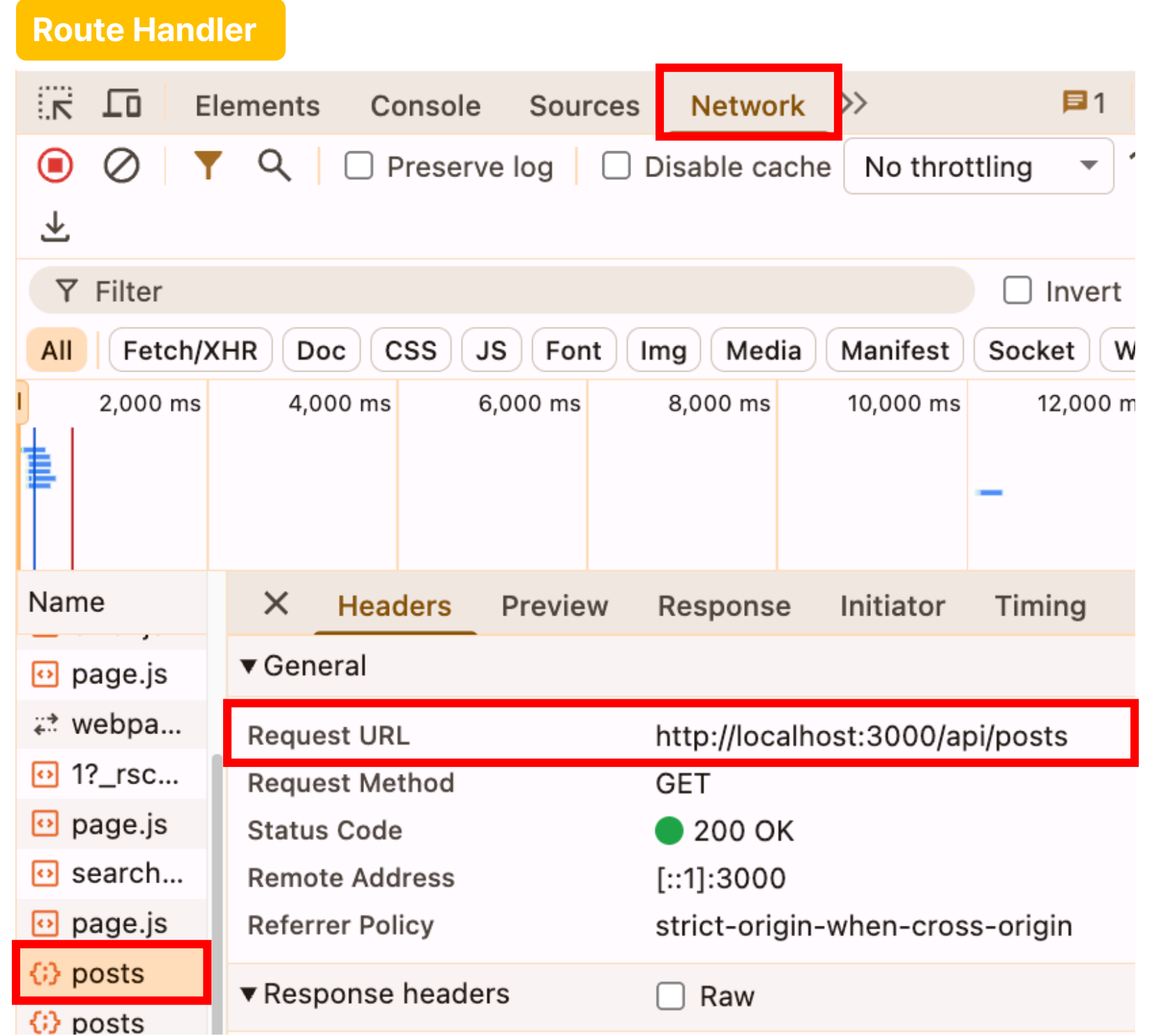
Direct Fetch



Network tab showing request details for 'posts'.

Name	Request URL	Request Method	Status Code	Remote Address	Referrer Policy
posts	http://localhost:8000/posts	GET	200 OK	127.0.0.1:8000	strict-origin-when-cross-origin

Route Handler



Network tab showing request details for 'posts'.

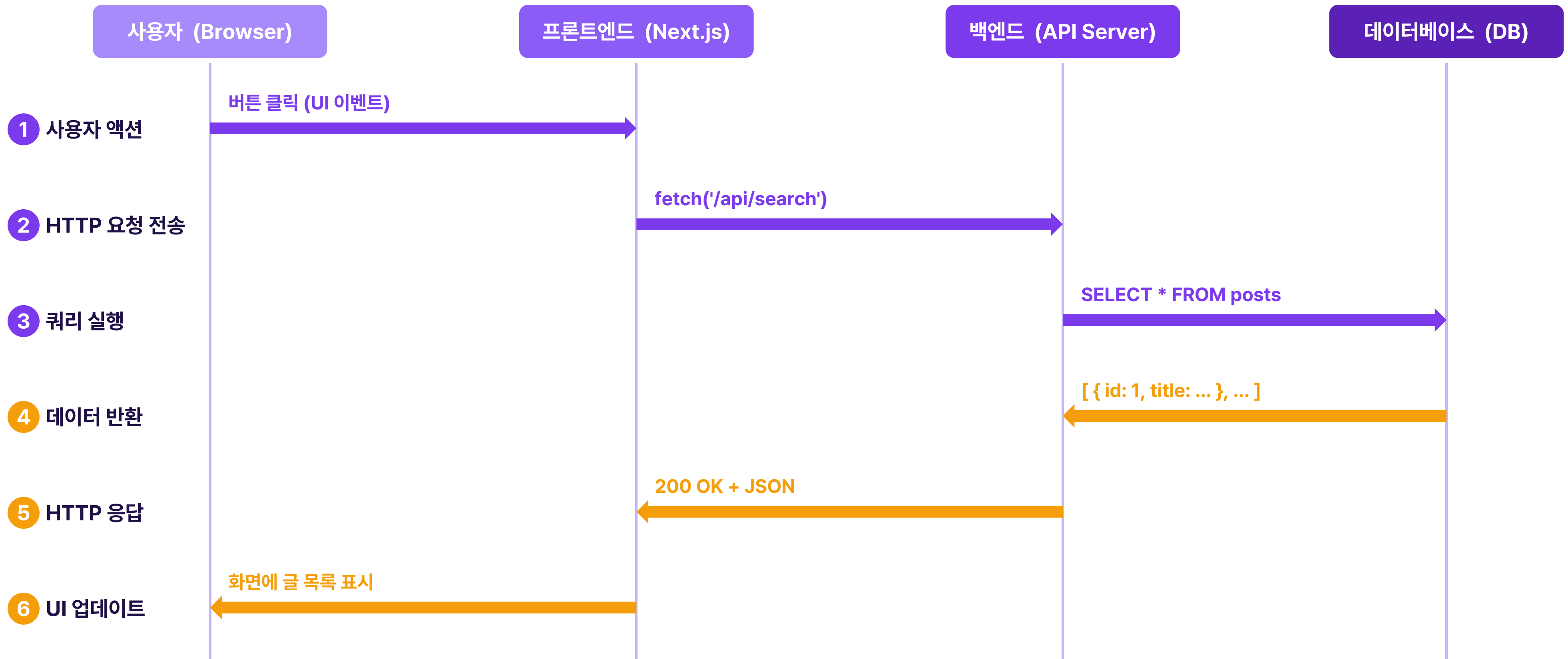
Name	Request URL	Request Method	Status Code	Remote Address	Referrer Policy
posts	http://localhost:3000/api/posts	GET	200 OK	:::1:3000	strict-origin-when-cross-origin

Direct Fetch와 Route Handler 한 눈에 비교하기

	Direct Fetch	Route Handler
요청 흐름	브라우저가 FastAPI 호출	브라우저 -> Next.js -> FastAPI
fetchURL	http://localhost:8000/posts	/api/posts
환경변수	NEXT_PUBLIC_FASTAPI_URL 필요	FASTAPI_URL 만으로 충분
백엔드 URL	브라우저 네트워크 탭에 노출	노출 안됨
CORS	FastAPI에서 허용 오리진 설정 필요	불필요
코드 복잡도	낮음	높음 (route.ts 파일 생성)
적절한 상황	공개 API 또는 간단한 프로토타입	API 키 등 민감한 정보 보호해야 하는 경우 응답 데이터 가공이 필요한 경우

- **Direct Fetch**는 단순하지만 백엔드 URL이 노출된다.
- **Route Handler**는 한 단계 더 거치는 대신 URL과 민감 정보를 숨길 수 있다.

프론트엔드 ↔ 백엔드 전체 흐름 다이어그램



- 요청 (Request) — 사용자 → DB 방향
- 응답 (Response) — DB → 사용자 방향

모든 요청은 반드시 응답과 짝을 이루며, 각 단계는 HTTP라는 약속 위에서 동작한다.
프론트엔드는 데이터를 "요청"하고, 백엔드는 그 데이터를 "가공·전달"하는 역할을 한다.

[실습] axios로 검색 기능 리팩토링하기

[실습] axios로 검색 페이지 리팩토링하기

브라우저와 Node.js에서 HTTP 요청을 보낼 수 있는 JavaScript 라이브러리



axios 응답 스키마

```
{
  data: {},
  status: 200,
  statusText: 'OK',
  headers: {},
  config: {},
  request: {}
}
```

```
const res = await axios.get<Post[]>('/posts');
console.log(res.data);
console.log(res.status);
console.log(res.headers)
```

- 자동 JSON 변환을 지원하여 **res.json()** 호출 없이 **res.data**로 접근 가능하다.
- 제네릭 타입을 지원하여 타입 안정성을 확보할 수 있다.

[실습] axios로 검색 페이지 리팩토링하기

axios 에러 스키마

property	정의
message	에러 요약 메시지와 실패한 status
name	에러 발생 위치 — axios 에러는 항상 'AxiosError'
stack	에러 스택 트레이스
config	요청 당시의 axios 설정 객체
code	axios 내부 에러 코드
status	HTTP 응답 상태 코드

```
try {  
  const res = await axios.get<Post[]>('/posts');  
  setPosts(res.data);  
} catch (error) {  
  if (axios.isAxiosError(error)) {  
    console.log(error.message);  
    console.log(error.name);  
    console.log(error.stack);  
    console.log(error.config?.url);  
    console.log(error.code);  
    console.log(error.response?.status);  
  }  
}
```

자동 에러 처리를 지원하여 **res.ok** 확인 없이 **4xx·5xx** 에러 핸들링이 가능하다.

[실습] axios로 검색 페이지 리팩토링하기

axios 에러 타입 및 코드

code	설명
ERR_BAD_OPTION_VALUE	axios 설정에 잘못된 값이 입력됨
ERR_BAD_OPTION	axios 설정에 유효하지 않은 옵션이 입력됨
ERR_NOT_SUPPORT	현재 환경에서 지원하지 않는 기능
ERR_DEPRECATED	더 이상 사용되지 않는 기능
ERR_INVALID_URL	유효하지 않은 URL
ECONNABORTED	요청 timeout 또는 브라우저에 의해 중단됨
ERR_CANCELED	AbortSignal 등으로 요청이 명시적으로 취소됨
ETIMEDOUT	axios 제한 시간 초과로 요청 timeout
ERR_NETWORK	네트워크 오류 — 브라우저에서는 CORS 위반도 포함
ERR_FR_TOO_MANY_REDIRECTS	리다이렉트 횟수가 설정 한도를 초과함
ERR_BAD_RESPONSE	응답 파싱 실패 또는 예상치 못한 형식 (주로 5xx)
ERR_BAD_REQUEST	잘못된 요청 형식 또는 필수 파라미터 누락 (주로 4xx)

- 자동 에러 처리를 지원하여 **res.ok** 확인 없이 **4xx·5xx** 에러 핸들링이 가능하다.
- 실무에서 주로 4가지 (**CORS, 4xx, 5xx, timeout**) 자주 발생한다.

axios 에러 핸들링

```
try {
  const res = await axios.get<Post[]>('/posts');
  setPosts(res.data);
} catch (error) {
  if (axios.isAxiosError(error)) {
    setError(error.response?.data?.message ?? '요청 중 오류가 발생했습니다. ');
  } else {
    setError('알 수 없는 오류가 발생했습니다. ');
  }
}
```

- 자동 에러 처리를 지원하여 **res.ok** 확인 없이 **4xx·5xx** 에러 핸들링 가능
- **axios.isAxiosError()**로 **axios** 에러 여부 구분 가능

[실습] axios로 검색 페이지 리팩토링하기

axios 메서드 별 문법

메서드	용도	포맷	예시
GET	조회	axios.get<T>(url)	<code>const res = await axios.get<Post[]>('/posts');</code>
POST	생성	axios.post<T>(url, data)	<code>const res = await axios.post<Post>('/posts', { title: '새 글 제목', body: '내용입니다', });</code>
PUT	전체 수정	axios.put<T>(url, data)	<code>const res = await axios.put<Post>('/posts/1', { title: ' 제목 수정', body: '내용 수정', });</code>
PATCH	일부 수정	axios.patch<T>(url, data)	<code>const res = await axios.patch<Post>('/posts/1', { title: '제목만 바꿀 수 있어요', });</code>
DELETE	삭제	axios.delete(url)	<code>await axios.delete('/posts/1');</code>

[실습] axios로 검색 페이지 리팩토링하기

axios 라이브러리 설치하기

↓ 아래 명령어로 axios를 설치해주세요.

```
$ npm install axios
```

package.json

```
{
  "name": "frontend",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint"
  },
  "dependencies": {
    "axios": "^1.16.0",
    "next": "^15.5.18",
    "react": "^19.0.0",
    "react-dom": "^19.0.0"
  },
  ...
}
```

터미널에서 **axios**를 설치한 후, **package.json** 파일에서 설치된 라이브러리를 확인한다.

[실습] axios로 검색 페이지 리팩토링하기

axios를 활용해 Route Handler 리팩토링하기

BEFORE

기존: fetch로 데이터 조회

```
useEffect(() => {
  setLoading(true);
  setError(null);

  fetch(`${BASE_PATH}/api/search`)
    .then((res) => {
      if (!res.ok) throw new Error("게시글을 불러오는 데 실패했습니다");
      return res.json();
    })
    .then((data: Post[]) => setResults(data))
    .catch((err: Error) => setError(err.message))
    .finally(() => setLoading(false));
}, []);
```



AFTER

변경: axios로 리팩토링

```
useEffect(() => {
  async function fetchPosts() {
    setLoading(true);
    setError(null);

    try {
      const res = await axios.get<Post[]>(`${BASE_PATH}/api/search`);
      setResults(res.data);
    }
    // ...
  }, [])
```

- **axios**를 활용해 **Route Handler**를 호출하도록 리팩토링한다.
- **Fetch**에서처럼 **res.json()**을 호출하지 않고 바로 데이터에 접근할 수 있다.
- **axios**가 지원하는 에러인지 확인하여 에러 핸들링한다.

[실습] axios로 검색 페이지 리팩토링하기

axios를 활용해 Route Handler 리팩토링하기

BEFORE

기존: fetch로 데이터 조회

```
export default function DeleteButton({ postId }: { postId: number }) {
  const BASE_PATH = process.env.NEXT_PUBLIC_BASE_PATH ?? "";

  async function handleDelete() {
    if (!confirm("정말 삭제할까요?")) return;

    const res = await fetch(`${BASE_PATH}/api/posts/${postId}`, {
      method: "DELETE",
    });

    if (res.ok) {
      window.location.href = `${BASE_PATH}/posts`;
    }
  }
  // ...
}
```



AFTER

변경: axios로 리팩토링

```
"use client";

import axios from "axios";

export default function DeleteButton({ postId }: { postId: number }) {
  const BASE_PATH = process.env.NEXT_PUBLIC_BASE_PATH ?? "";

  async function handleDelete() {
    if (!confirm("정말 삭제할까요?")) return;

    try {
      await axios.delete(`${BASE_PATH}/api/posts/${postId}`);
      window.location.href = `${BASE_PATH}/posts`;
    } catch (err) {
      if (axios.isAxiosError(err)) {
        alert(err.response?.data?.detail ?? "게시글 삭제에 실패했습니다");
      }
    }
  }
  // ...
}
```

DeleteButton.tsx도 **axios**를 활용해 호출하도록 리팩토링한다.

[실습] axios로 검색 페이지 리팩토링하기

Fetch와 axios 방식 비교

frontend/app/search/page.tsx

```
useEffect(() => {
  async function fetchPosts() {
    setLoading(true);
    setError(null);

    try {
      const res = await fetch("/api/posts");

      if (!res.ok) {
        throw new Error("게시글 목록을 불러오는 데 실패했습니다");
      }

      const data: Post[] = await res.json();
      setResults(data);
    }
  }
});
```



frontend/app/search/page.tsx

```
useEffect(() => {
  async function fetchPosts() {
    setLoading(true);
    setError(null);

    try {
      const res = await axios.get<Post[]>(`${BASE_PATH}/api/search`);
      setResults(res.data);
    } catch (err) {
      if (axios.isAxiosError(err)) {
        setError(err.response?.data?.detail ?? "게시글을 불러오는 데 실패했습니다");
      } else {
        setError("알 수 없는 오류가 발생했습니다");
      }
    } finally {
      setLoading(false);
    }
  }

  fetchPosts();
}, [])
```

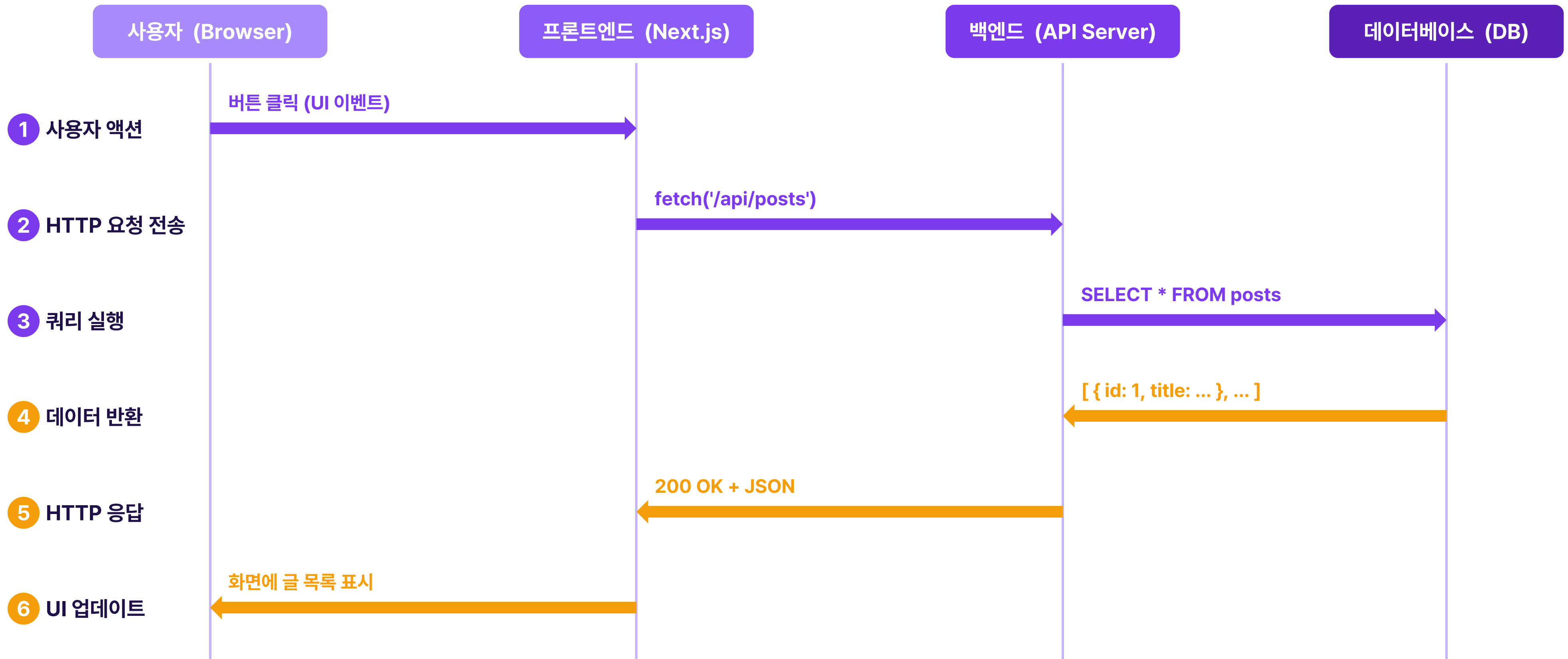
Route Handler(route.ts)는 서버에서 실행되어 Node.js 내장 fetch로 충분하므로,
axios는 클라이언트에만 적용한다.

Fetch vs axios 비교표

항목	fetch	axios
설치	브라우저 내장 (별도 설치 불필요)	<code>npm install axios</code> 필요
JSON 변환	<code>res.json()</code> 직접 호출 필요	<code>res.data</code> 로 자동 변환
에러 처리	<code>res.ok</code> 로 직접 체크 필요	<code>4xx·5xx</code> 자동으로 catch 블록으로 넘어감
타입 지원	별도 타입 지정 어려움	제너릭 지원

[실습] 검색 기능 고도화하기

프론트엔드 ↔ 백엔드 전체 흐름 다이어그램



- 요청 (Request) — 사용자 → DB 방향
- 응답 (Response) — DB → 사용자 방향

모든 요청은 반드시 응답과 짝을 이루며, 각 단계는 HTTP라는 약속 위에서 동작한다.
프론트엔드는 데이터를 "요청"하고, 백엔드는 그 데이터를 "가공·전달"하는 역할을 한다.

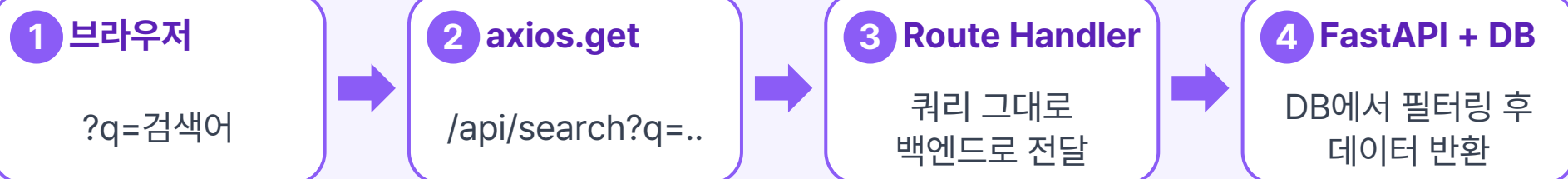


[실습] 검색 기능 고도화하기

AFTER · 변경 방식

URL 쿼리 파라미터로 검색어를 보내고 서버에서 필터링

데이터 흐름 · 서버가 필터링까지 책임짐



핵심 특징

- URL이 검색 상태를 표현하며 공유 · 북마크 · 뒤로가기 가능
- 필요한 결과만 전송하며 데이터가 많아도 가벼움
- 정렬 · 페이지네이션도 같은 패턴으로 확장 가능

변경으로 얻는 효과

공유 가능

URL이 곧 검색 상태

검색 결과 URL을 복사해 공유하거나 북마크할 수 있고, 뒤로가기 · 앞으로 가기도 자연스럽게 동작한다.

성능

필요한 만큼만 전송

DB가 미리 필터링해 결과만 보내므로 네트워크·렌더링 부담이 줄고, 데이터가 커져도 화면이 가볍다.

확장성

같은 패턴으로 기능 추가

정렬 · 페이지네이션 · 필터를 동일한 URL 파라미터 방식으로 확장할 수 있고, 권한 · 캐싱도 서버에서 처리할 수 있다.



[실습] 검색 기능 고도화하기

backend/main.py

BEFORE

기존: 항상 전체 게시물 반환

```
@app.get("/posts",
response_model=list[PostResponse])
def get_posts(db: Session =
Depends(get_db)):
    return
    db.execute(select(Post)).scalars().all()
```



AFTER

변경: q 파라미터로 SQL 필터링

```
@app.get("/posts", response_model=list[PostResponse])
def get_posts(
    q: Optional[str] = None,
    db: Session = Depends(get_db),
):
    if q:
        stmt = select(Post).where(
            or_(Post.title.contains(q), Post.content.contains(q))
        )
    else:
        stmt = select(Post)
    return db.execute(stmt).scalars().all()
```

- `get_posts` 함수에 `q: Optional[str] = None` 인자를 선언하면 `/posts?q=검색어` 형태로 쿼리 파라미터를 받을 수 있다.
- `q`가 없으면 기존과 동일하게 전체 게시글을 반환한다.
- `q`가 있으면 SQLAlchemy `or` 조건을 적용하여 제목 또는 본문 내용으로 필터링을 적용한다.



[실습] 검색 기능 고도화하기

frontend/app/api/search/route.ts

BEFORE

기존: 모든 게시물 정보를 조회

```
export async function GET() {  
  const res = await fetch(`${process.env.FASTAPI_URL}/posts`);  
  
  if (!res.ok) {  
    return Response.json(  
      { detail: "게시글 목록을 불러오는 데 실패했습니다" },  
      { status: res.status }  
    );  
  }  
  
  const data = await res.json();  
  return Response.json(data);  
}
```



AFTER

변경: 쿼리 파싱 후 FastAPI URL 분기

```
export async function GET(request: Request) {  
  const { searchParams } = new URL(request.url);  
  const q = searchParams.get("q");  
  const url = q  
    ? `${process.env.FASTAPI_URL}/posts?q=${encodeURIComponent(q)}`  
    : `${process.env.FASTAPI_URL}/posts`;  
  
  const res = await fetch(url);  
  
  if (!res.ok) {  
    return Response.json(  
      { detail: "게시글 목록을 불러오는 데 실패했습니다" },  
      { status: res.status }  
    );  
  }  
  
  const data = await res.json();  
  return Response.json(data);  
}
```

- **GET(request: Request)**로 시그니처를 변경해 브라우저의 요청 객체를 직접 받는다
- **new URL(request.url)**로 URL을 파싱한 뒤 **searchParams.get("q")**로 검색어를 추출한다.
- **q** 유무에 따라 있으면 **/posts?q=검색어**, 없으면 **/posts** FastAPI 호출 URL을 분기한다.
- **encodeURIComponent**를 사용하여 검색어에 한글 공백 특수문자가 포함된 경우 안전한 형태로 인코딩한다.



[실습] 검색 기능 고도화하기

encodeURIComponent

BEFORE

기존: 한글과 공백, 특수문자가 URL에 그대로 포함

```
const q = "Next.js 입문"

const url = `${process.env.FASTAPI_URL}/posts?q=${q}`
```

```
http://localhost:8000/posts?q=Next.js 입문
```

AFTER

변경: 한글과 공백, 특수문자가 인코딩되어 URL에 포함

```
const q = "Next.js 입문"

const url = `${process.env.FASTAPI_URL}/posts?
q=${encodeURIComponent(q)}`
```

```
http://localhost:8000/posts?q=Next.js%20%EC%9E%85%EB%AC%B
```

- 공백이나 한글, 특수문자가 포함된 URL은 브라우저/서버가 어디서 파라미터가 끊기는지 판단하지 못한다.
- **Route Handler**에서 **FastAPI**로 URL을 직접 조합할 때, **encodeURIComponent**로 인코딩하여 URL이 깨지지 않도록 보장해야 한다.



[실습] 검색 기능 고도화하기

frontend/app/search/page.tsx

BEFORE

기존: 받은 데이터를 클라이언트에서 filter()

```
const filtered = results.filter(post =>
  post.title.toLowerCase().includes(query.toLowerCase()) ||
  post.content.toLowerCase().includes(query.toLowerCase())
);
```

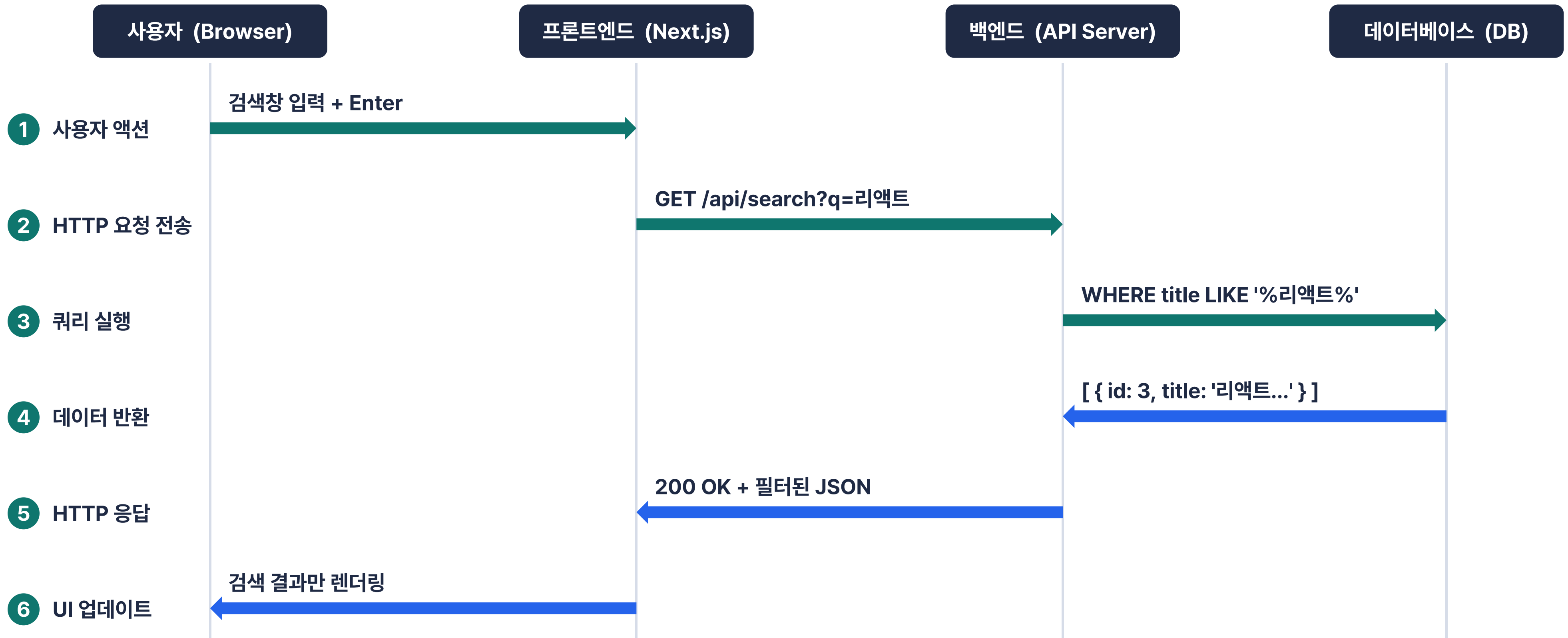
AFTER

변경: URL 쿼리로 검색, 서버 필터링 결과만 수신

```
// const filtered = results.filter(
//   (post) =>
//     post.title.toLowerCase().includes(query.toLowerCase()) ||
//     post.content.toLowerCase().includes(query.toLowerCase())
// );
```

- **useSearchParams()**로 현재 URL의 **q** 값을 읽어 검색어 상태로 사용한다.
- 입력창 **onChange** 시 **router**를 **활용해** URL을 갱신한다.
- 클라이언트 **.filter()** 로직을 제거하고 데이터 처리는 백엔드로 위임한다.

프론트엔드 ↔ 백엔드 전체 흐름 다이어그램



- 요청 (Request) — 검색어를 서버로 전달
- 응답 (Response) — 필터된 결과만 반환

검색어를 URL에 담아 서버로 보내면, 백엔드가 DB에서 직접 필터링한 결과만 돌려준다.
URL이 곧 검색 상태이므로 공유·북마크·뒤로가기가 자연스럽고, 브라우저는 필요한 데이터만 받는다.